An invitation to univalent foundations

of mathematics and computer science

Dr Jon Sterling

Associate Professor in Logical
Foundations and Formal Methods

Computer Laboratory
University of Cambridge

Wednesday Seminar
25 October 2023

What is a “foundation”?

A **foundation** concentrates prior mathematical experience into a *language*. Like any human language, a foundation has both a *grammar* and a *vocabulary*, which can vary greatly in their fidelity to a given sphere of mathematical experience.

Different approaches to foundations

The *grammar* of mathematics can be organised in many ways.

Material set theory: first-order logic over a theory with a *single* type of entity (sets), formulas built up from the $\in$-relation. Example: ZFC.

Categorical set theories: first-order logic over a theory with two types: sets and functions. Example: ETCS.

Simple type theory / HOL: first-order logic over a theory with many types, including a type of propositions. Example: Isabelle/HOL.

Dependent type theories: algebraic(*) theories allowing many types, each corresponding to a different kind of mathematical entity (*e.g*. integers, reals, automata, vector spaces, propositions, *etc*.). Examples: Martin-Löf Type Theory, HoTT, Calculus of Constructions.

Analytic *vs*. synthetic theories of sets

Mathematical practice is far-removed from the vocabulary of set theoretic foundations — for instance, when was the last time you mentioned the "axiom of regularity" in a proof? Even the axioms of "pairing" and "union" are deeply impertinent...

Everyday mathematical practice works at a higher level of abstraction. The basic move are things like:

1. The product of a family of sets is a set.
2. The disjoint sum of a family of sets is a set.
3. The quotient of a set by an equivalence relation is a set.

Type theory takes the above (and beyond) as its *primitive* rules.

Whereas collections in set theory are built analytically from $\in$-trees, type theory delivers a **synthetic theory of collections**.

Primer on (dependent) type theory

There are two kinds of entity in type theory: *types* and *elements*. (*Sets* will be described later as special types.)

$$\boxed{A : \mathbf{Type}} \qquad\qquad \boxed{u : A}$$

The colon is *not* a predicate: it is an annotation. Just as in Java or Haskell, the type of an element is part of that element.

The vocabulary of type theory unfolds by specifying rules to form (new) types and elements.

The vocabulary of type theory

Let $A : \mathbf{Type}$ be a type, and let $B(x) : \mathbf{Type}$ be a family of types indexed in $x : A$. Then:

1. The product $\prod_{(x:A)} B(x)$ is a type; an element of $\prod_{(x:A)} B(x)$ is given by a function $f(x) : B(x)$ varying in $x : A$.
2. The disjoint sum $\sum_{(x:A)} B(x)$ is a type; an element of $\sum_{(x:A)} B(x)$ is given by a pair (u, v) where $u : A$ and $v : B(u)$.

We also have a type $\mathbb{N} : \mathbf{Type}$ of natural numbers, defined inductively by two constructors: zero and successor.

Equality and identification in type theory

In traditional mathematics, there is an *equality* predicate $x = y$ defined for any entities x, y of the same type.

The principle of equivalence. The important thing about equality is that respected by *everything*! If $x = y$ then $P(x) = P(y)$.

Type theory takes (roughly) the above as a *definition* of what it means for two entities to be the same. **In type theory, this principle is compatible with more than just traditional equality!**

The principle of equivalence and univalent foundations

In type theory, traditional equality is *not* the only thing that is respected by all expressible notions!

"Univalence": If $f : A \to B$ is a bijective function, then any expressible property of A also holds of B.

Univalent type theory takes the above principle of univalence as a basic axiom, alongside the axioms of products and disjoint sums, *etc*.

Achieved by replacing equality with a more general and more flexible notion of "identification" that includes equality, isomorphism, and more.

Identification types

Fix a type $A : \mathbf{Type}$.

1. **Formation.** Given elements $x, y : A$ we may form the *identification type* $x =_A y : \mathbf{Type}$.
2. **Construction.** Given $x : A$ we may form the *reflexivity identification* $\mathrm{refl}_x : x =_A x$.
3. **Induction.** Given $x : A$, to define a family of functions $f(y, p) : B(y, p)$ varying in $y : A$ and $p : x =_A y$, it suffices to define $p(x, \mathrm{refl}_x) : B(x, \mathrm{refl}_x)$.

Identification induction

Induction. *Given* $x : A$*, to define a family of functions* $f(y, p) : B(y, p)$ *varying in* $y : A$ *and* $p : x =_A y$*, it suffices to define* $p(x, \mathrm{refl}_x) : B(x, \mathrm{refl}_x)$*.*

$$\text{symmetry} : \prod_{(x,y:A)} \prod_{(p:x=_A y)} y =_A x$$
$$\text{symmetry}\, x\, x\, \mathrm{refl}_x :\equiv \mathrm{refl}_x$$

$$\text{transitivity} : \prod_{(x,y:A)} \prod_{(p:x=_A y)} \prod_{(z:A)} \prod_{(q:y=_A z)} x =_A z$$
$$\text{transitivity}\, x\, x\, \mathrm{refl}_x\, z\, q :\equiv q$$

$$\text{congruence} : \prod_{(x,y:A)} \prod_{(p:x=_A y)} \prod_{(f:A\to B)} fx =_B fy$$
$$\text{congruence}\, x\, x\, \mathrm{refl}_x\, f :\equiv \mathrm{refl}_{fx}$$

But you cannot prove: $\prod_{(x,y:A)} \prod_{(p,q:x=_A y)} p =_{x=_A y} q$, except in the case of certain types A (which will be called "sets").

Propositions and sets in univalent foundations

Identification types are used to isolate types that behave like *propositions (truth values)* and like *sets* from traditional foundations.

Definition

A type $A : \mathbf{Type}$ is called a ***proposition*** when any two of its elements may be identified, *i.e.* we have:

$$\mathrm{isProp}(A) :\equiv \prod_{(x,y:A)} x =_A y$$

Definition

A type $A : \mathbf{Type}$ is called a ***set*** when for any $x, y : A$ the identification type $x =_A y$ is a proposition, *i.e.* we have:

$$\begin{aligned}\mathrm{isSet}(A) &:\equiv \prod_{(x,y:A)} \mathrm{isProp}(x =_A y)\\ &\equiv \prod_{(x,y:A)} \prod_{(p,q:x=_A y)} p =_{x=_A y} q\end{aligned}$$

Non-propositions and non-sets in univalent foundations

The simplest set that is not a proposition is $\mathbf{2} \equiv \{0, 1\}$. For a type that is not a set, we may consider the type **Set** of all (small) sets!

In univalent foundations, identifications between sets are given by *isomorphisms*. **Set** fails to be a set *not* because of size paradoxes, but because there can be more than one isomorphism between two sets.

Characterizing identification types in univalent foundations

The built-in notion of identification is global, just like equality in set theory. For any specific kind of object (like sets, groups, *etc*.) we may prove a *characterization theorem* that makes the structure of identifications explicit. For example:

1. Identifications between functions $f, g : A \to B$ are given by *homotopies* $h : \prod_{(x:A)} fx =_A gx$.
2. Identifications between sets are given by isomorphisms.
3. Identifications between groups are given by *invertible group homomorphisms*.
4. Identifications between categories are given by *categorical equivalences* (full, faithful, and essentially surjective functors).

All these characterizations follow from the axioms.

Univalent foundations of computer programming

The system I have so far described is secretly a *programming language*. We can *run* the code and even analyze its complexity.

Dependent types handle two things:

1. Parameterization of program *modules* in their dependencies.
2. Specification and verification of functional correctness using induction and equational reasoning.

The impact of *univalence* is to expand the power of equational reasoning to include representation independence.

A programming example: functional queues

In univalent foundations, we may describe the type of *functional queues* as an iterated disjoint sum — which we write as a "record".

$$
\begin{aligned}
&\textbf{record}\ \mathrm{QueueStructure} : \textbf{Type} :\equiv \\
&\quad \mathit{queue} : \textbf{Set} \to \textbf{Set} \\
&\quad \mathit{init} : \textstyle\prod_{(A:\textbf{Set})} \mathit{queue}\, A \\
&\quad \mathit{enqeue} : \textstyle\prod_{(A:\textbf{Set})} A \times \mathit{queue}\, A \to \mathit{queue}\, A \\
&\quad \mathit{dequeue} : \textstyle\prod_{(A:\textbf{Set})} \mathit{queue}\, A \to \mathrm{option}\,(A \times \mathit{queue}\, A) \\
&\quad \ldots
\end{aligned}
$$

A naïve queue implementation

We can define an implementation ListQueue : QueueStructure by taking to ListQueue.*queue* be the type of cons lists.

This is very easy to implement, but it has poor complexity: dequeuing is linear in the size of the list, as we must reach the back of the list.

A refactoring problem

Suppose we have written a program, *e.g*. a web server, that uses a queue structure as part of its implementation.

Because we were just prototyping, we started off with the naïve ListQueue implementation. Our server is *dying* under trivial loads!

As we only interacted with ListQueue through the operations specified by QueueStructure, it is not difficult to *abstract* our server into a function:

$$\mathit{server} : \mathsf{QueueStructure} \to \mathsf{WebServer}$$

If we implement a faster queue structure, we can plug it into our server. But how do we know that our server will still work?